

Sprint 1: Plan the Two Weeks

You have your team. You have your ideas from Project 1. This week you turn them into a plan. Next week you start building.

EC601 • Due in one week • One document (SPRINT1.md, 4 pages max) + your GitHub board

What goes in the plan

- 1. Mission.** One sentence, using the template from class. Do not call it final — it is your best guess until your first test says otherwise.
- 2. Target user.** One specific person. If your product has several users, name the primary one and build for them first.
- 3. User stories.** Your top 5, with acceptance criteria. Put them on your GitHub board as issues — the board is the plan; the document just explains it.
- 4. Feasibility — show me, don't tell me.** “We will use dataset X” is a wish. Downloaded, loaded by a script, committed to the repo — that is feasibility. Same for API keys and hardware: prove access this week.
- 5. Tooling.** Languages, frameworks, models, and why — one line each. Full setup: next slide.
- 6. The demo sentence.** “At the end of two weeks we will show _____ working end to end.” One sentence. If you cannot write it, your goals are too vague.

Everything above fits in 2 pages. Long plans hide unclear thinking — short plans expose it, which is the point.

GitHub and tools — day one, not day ten

Everything lives in one team repository. If it is not in the repo, it does not exist.

The repository. One repo per team. README that says what the project is and how to run it. SPRINT1.md, TEAM.md, and your documents in /docs. Add me and the TA as collaborators this week.

The board. GitHub Projects board with three columns: To do, Doing, Done. Every user story is an issue on it, assigned to a person. If I open your board, I should see who is doing what today.

How you write code together. No pushing to main. Branch per story, pull request to merge, and a teammate reviews before it goes in. Reviewing each other's code is the same skill you practiced on each other's papers.

Reproducible from day one. requirements.txt or environment file, committed. A .env.example for keys — never the keys themselves. Test: a teammate clones the repo on their machine and it runs. If it only runs on your laptop, it does not run.

List every tool, with a reason. Language, framework, model or API (and its cost or rate limit), data tools, testing tool, where it runs (laptop, SCC, cloud credits). One line each in SPRINT1.md. “The tutorial used it” is an honest reason. “It looked cool” is not a reason.

One communication channel. Slack, Discord, WhatsApp — I do not care which. Pick one, put the link in TEAM.md, and stop using the other four.

The check: I clone your repo, open your board, and understand your project without asking you anything.

Now the researcher part

7. Your riskiest assumption — and its test. List 3–5 assumptions your project stands on. Pick the one that kills the project if wrong. Test that one first, with the cheapest test you can find. Write your pivot line now: “We change direction if ...” — decide it while you are still honest.

8. Evaluation with a baseline. Name your metric AND what you compare against. “Our system finds relevant papers” is marketing. “Beats keyword alerts on recall, on a 50-paper labeled set” is research. No baseline = no claim. If you cannot name a baseline, you have not defined the task yet — that is a finding, write it down.

9. Related work, as a team. Each of you read papers for Project 1. Merge them: 8–10 papers, one line each — what it does, and what it leaves open for you. That open part is your project.

10. Three lines on harm. Who gets hurt if your system is wrong? What is the worst realistic misuse? Three lines now. We will go deeper in Project 3.

A good researcher is not the one who is right — it is the one who finds out fastest whether they are wrong.

First: how you work together

Teams rarely fail on technology. They fail on coordination. Write this page now, while nobody is upset — you cannot write fair rules in the middle of a fight.

Roles. Who owns what for this sprint. Rotate a sprint lead — everyone leads once this semester.

Cadence. When you meet, where you talk (one channel, not five), and how fast you answer each other.

Decisions. When two of you disagree, how does the team decide? Vote? Sprint lead calls it? Pick one now.

Silence. What happens when someone disappears for three days? Agree on the first step — it is a message to them, not a message to me.

Put it in the repo as TEAM.md. When there is a conflict later — and there will be — your agreement handles it before I do.

This is the page you will be most tempted to skip. It is also the one that saves your semester.

Two AIs review your plan — make them fight

One AI reviewing your plan will be polite. Two AIs arguing about it will be useful. Use two different models (for example Claude and ChatGPT) — not two chats with the same one.

Step 1. Give the same plan to both. Ask each the same three questions: What is our riskiest unstated assumption? Why might this fail by week 6? What is missing from our evaluation?

Step 2. Cross the reviews: give each model the other's critique. Ask it to attack or defend it.

Step 3. Write the short log: where they agreed (probably a real problem), where they disagreed (you decide, and say why), and the one change you made to the plan because of it.

If the review changed nothing, you did not review — you collected compliments. Chat links go in the repo, as always.

You are responsible for the outcome. The AIs advise; the team decides.

Find the faculty who can help you

List 2–3 BU professors whose work touches your project. Use the Research Areas handout as a start — then check their recent papers or lab page yourself. A name without a reason does not count.

For each one, write: why they are relevant (cite one paper or project of theirs), and one specific question you would ask them.

Make the question real. “Can you help us?” has no answer. “For a lab-alert tool, would you weight recall over precision?” gets a reply. If you cannot write the specific question, you do not understand your project yet.

If you email them (optional, encouraged): read their latest abstract first, keep it under 150 words, and ask for nothing that costs more than 15 minutes. Professors answer good questions. They delete vague ones.

Listing a professor does not obligate them — it prepares you. Knowing who to ask is half of research.

Deliverables in one week: SPRINT1.md (4 pages max) • TEAM.md • GitHub board • AI review log • faculty list.

Thank you